Advanced Data Structures
CS4580

Lecture 03

Double-Ended Priority Queues

- Primary operations
  - Insert
  - Remove Max
  - Remove Min
- Note that a single-ended priority queue supports just one of the above remove operations.

General Methods

- Dual min and max single-ended priority queues.
- Correspondence based min and max single-ended priority queues.

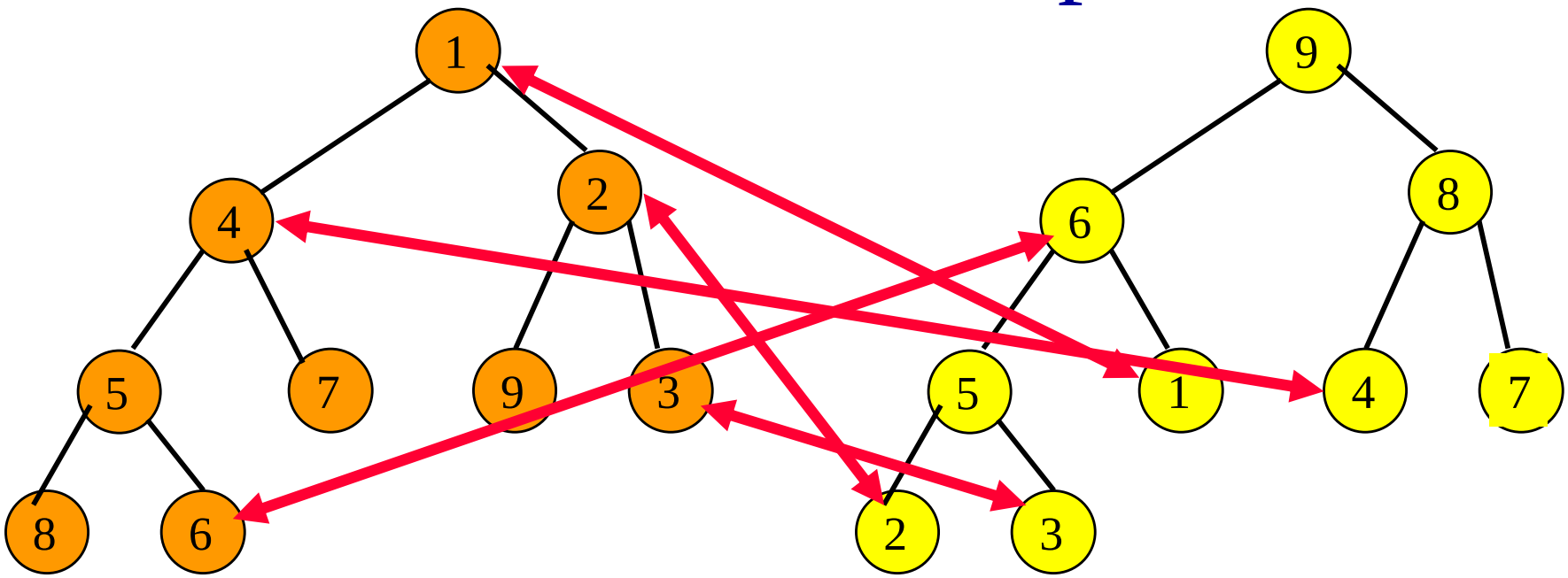
Specialized Structures

- Symmetric min-max heaps.
- Min-max heaps.
- Deaps.
- ✓Interval heaps.

Dual Single-Ended Priority Queues

- Each element is in both a min and a max single-ended priority queue.
- Single-ended priority queue also must support an arbitrary remove.
- Each node in a priority queue has a pointer to the node in the other priority queue that has the same element.

9-Element Example



Min Heap

Max Heap

- Only 5 of 9 two-way pointers shown.
- Insert, remove min, remove max, initialize.
- Operation cost is more than doubled relative to heap.
- Space for 2n nodes.

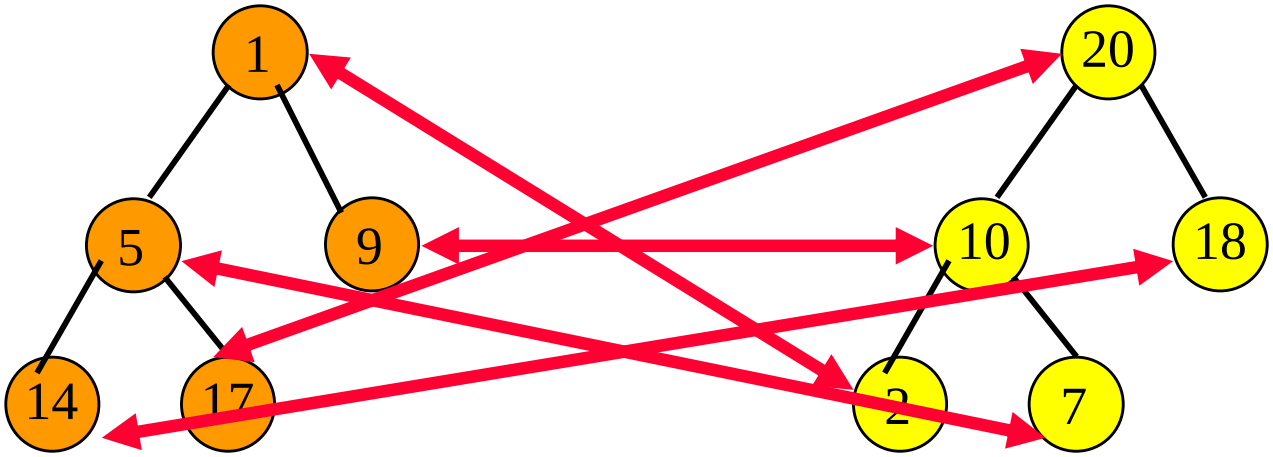
Correspondence Structures

- Use a min and a max single-ended priority queue, each has n/2 nodes (or elements).
- When n is odd, 1 element is in a buffer.
- Remaining elements are in the single-ended priority queues.
- Establish a correspondence between the min and max single-ended priority queues.
  - Total correspondence.
  - Leaf correspondence.
- Single-ended priority queue also must support an arbitrary remove.

Total Correspondence

- Each element of the min priority queue is paired with a different and >= element in the max priority queue.

Total Correspondence Example

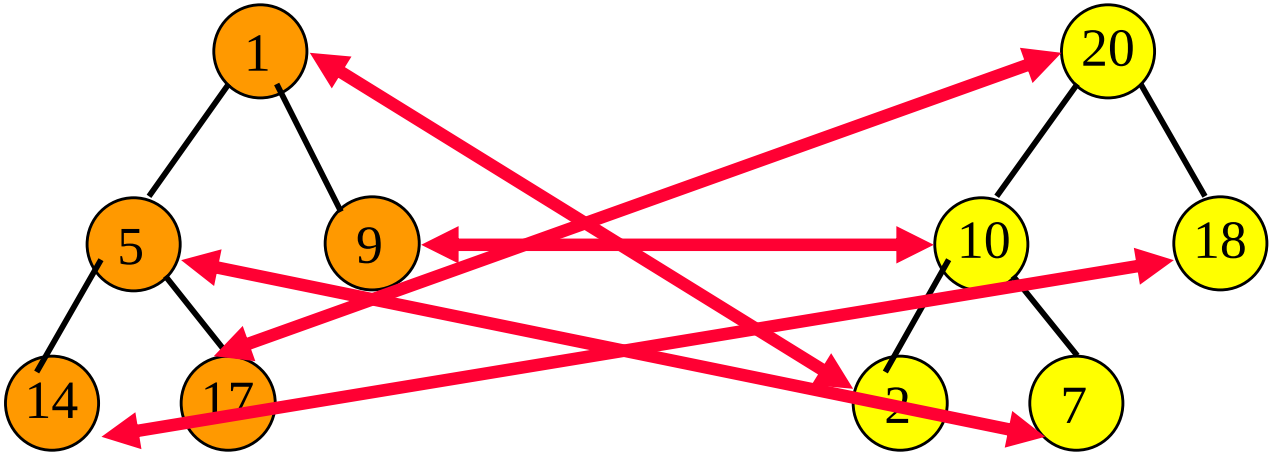


Min Heap

Max Heap

Buffer = 12

Insert



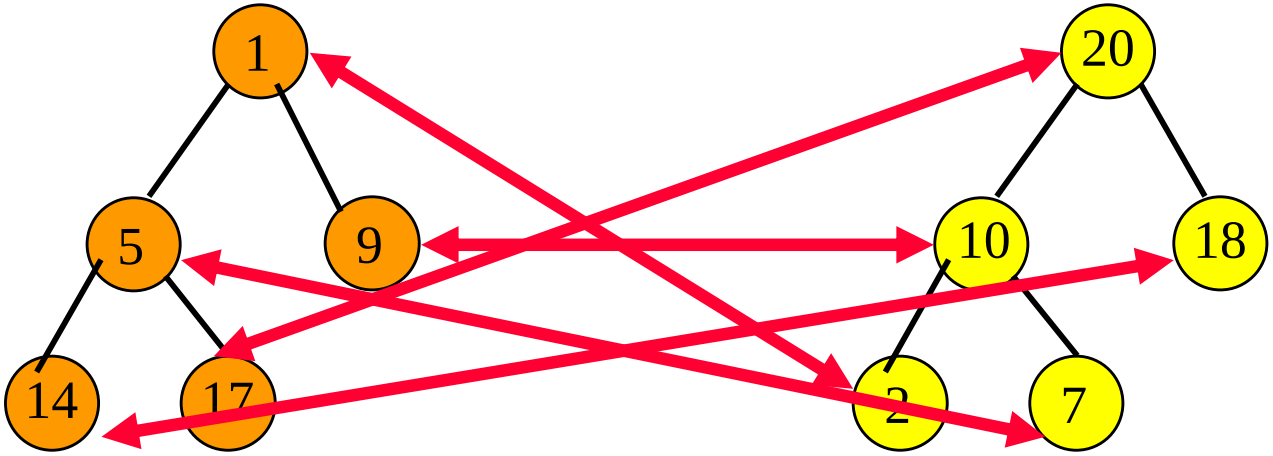
Min Heap

Max Heap

Buffer = 12

- Buffer empty => place in buffer.
- Else, insert smaller of new and buffer elements into min priority queue and larger into max priority queue; establish correspondence between the 2 elements.

Remove Min



Min Heap

Max Heap

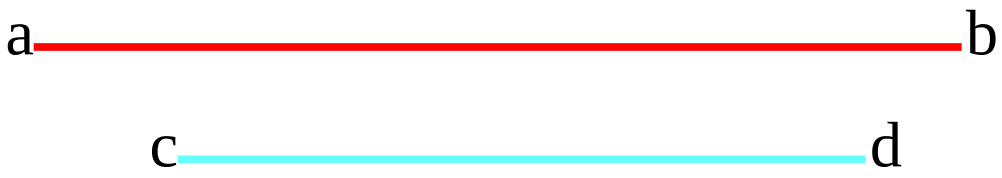
Buffer = 12

- Buffer is min => empty buffer.
- Else, remove min from min PQ as well as corresponding element from max PQ; reinsert corresponding element.

Interval Heaps

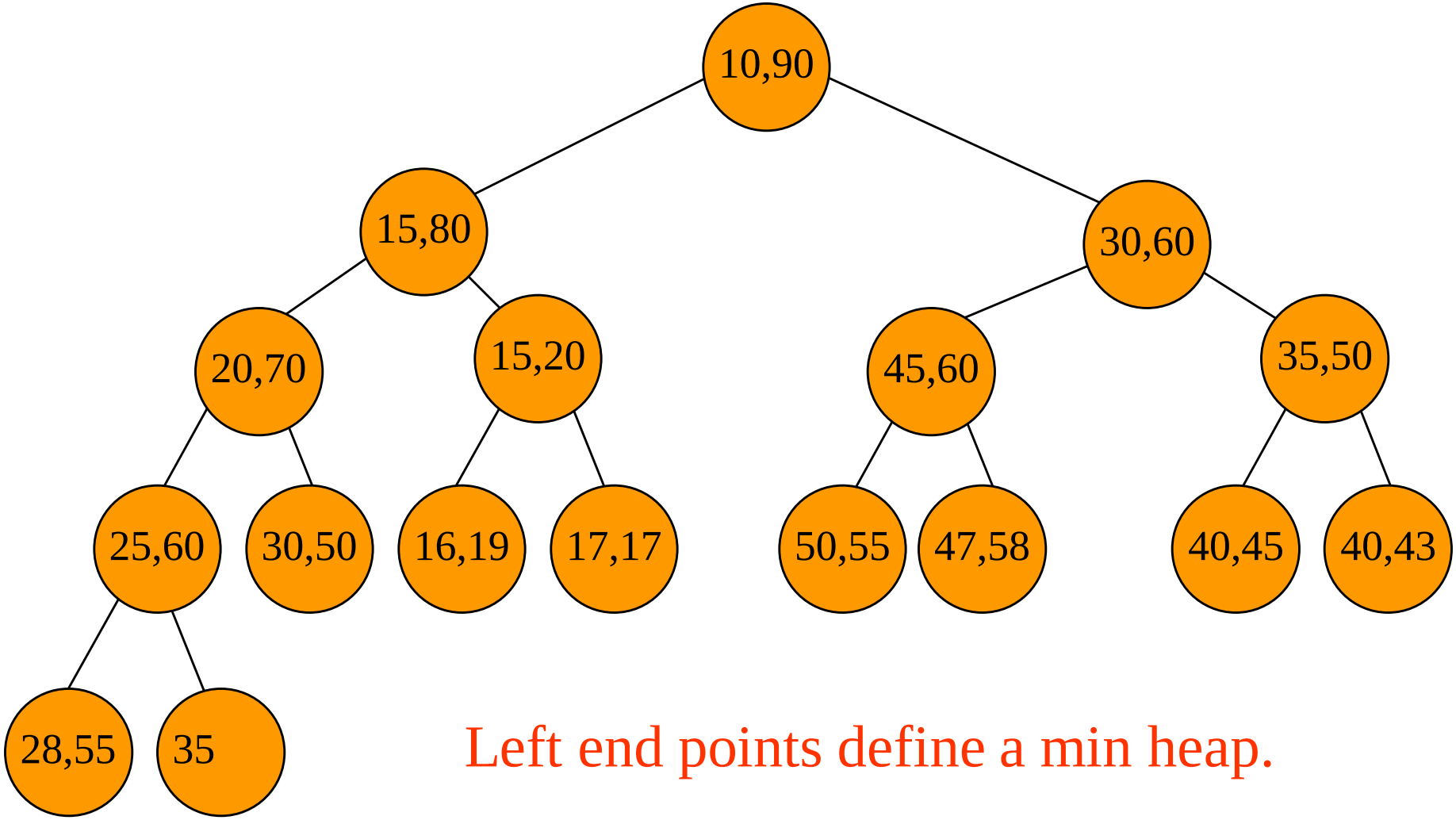
- Complete binary tree.
- Each node (except possibly last one) has 2 elements.
- Last node has 1 or 2 elements.
- Let a and b be the elements in a node P, a <= b.
- [a, b] is the interval represented by P.
- The interval represented by a node that has just one element a is [a, a].
- The interval [c, d] is contained in interval [a, b] iff a <= c <= d <= b.
- In an interval heap each node's (except for root) interval is contained in that of its parent.

Interval



- [c,d] is contained in [a,b]
- a <= c
- d <= b

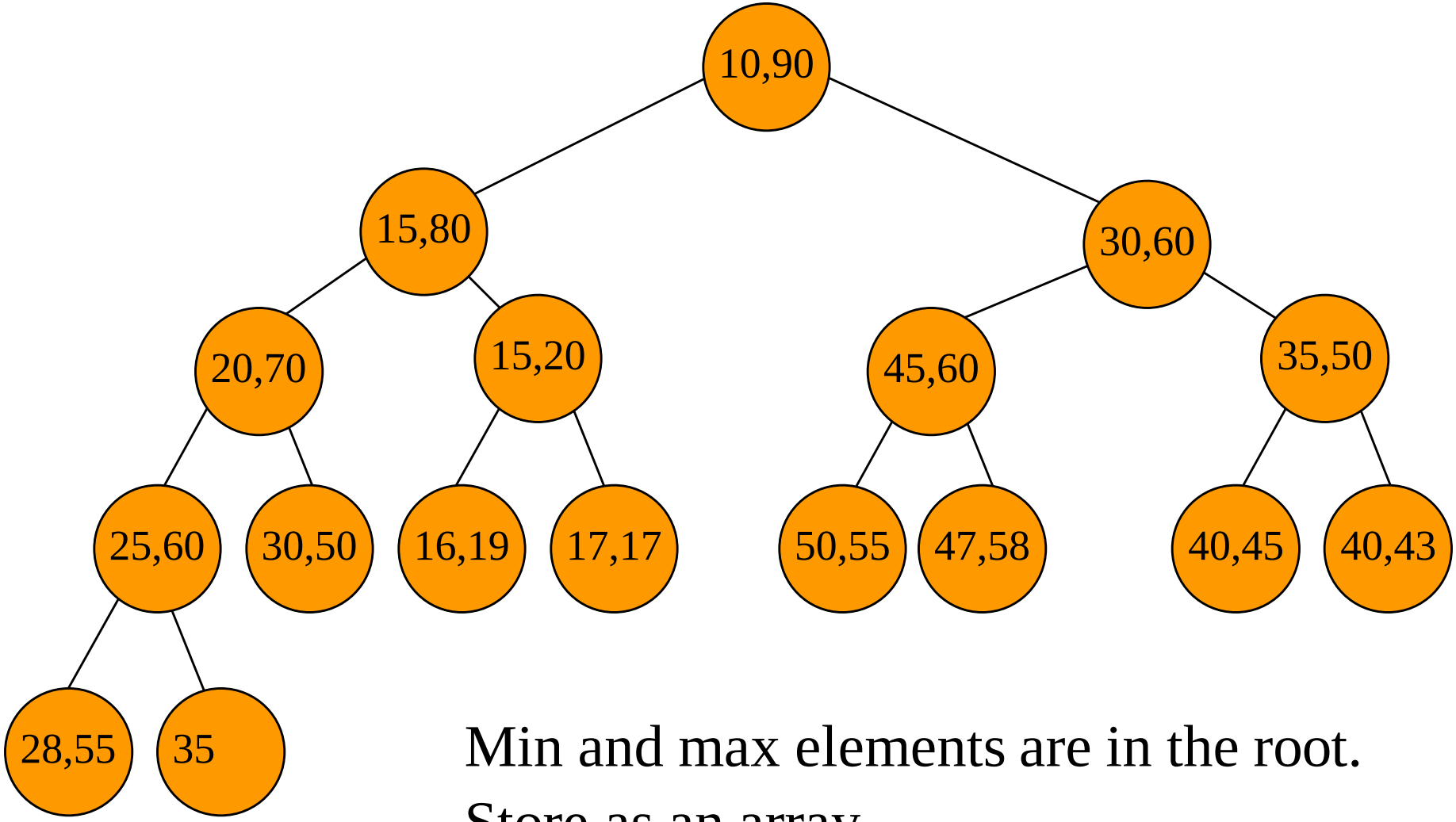
Example Interval Heap



Left end points define a min heap.

Right end points define a max heap.

Example Interval Heap

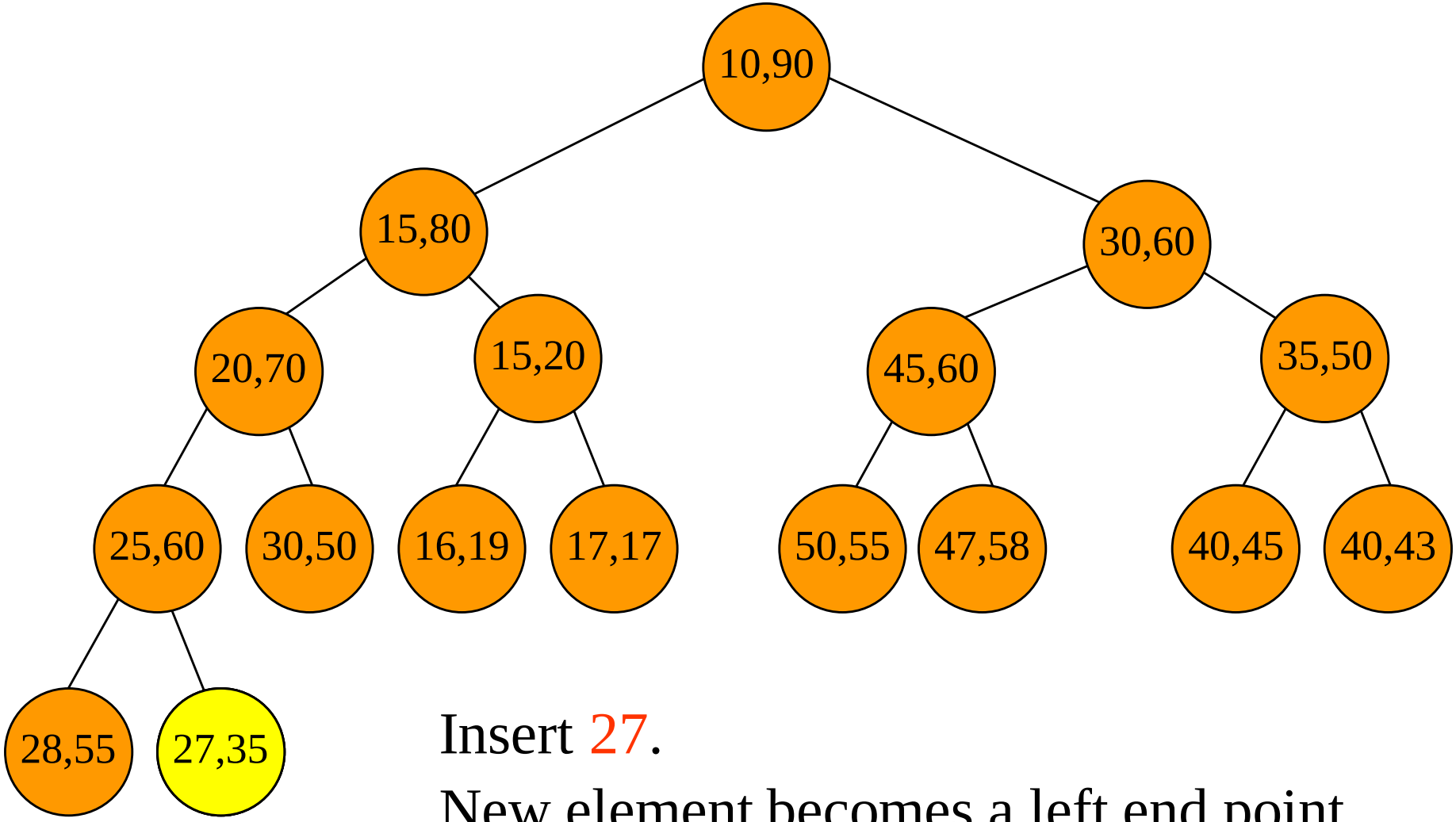


Min and max elements are in the root.

Store as an array.

Height is ~$\log_2 n$.

Insert An Element

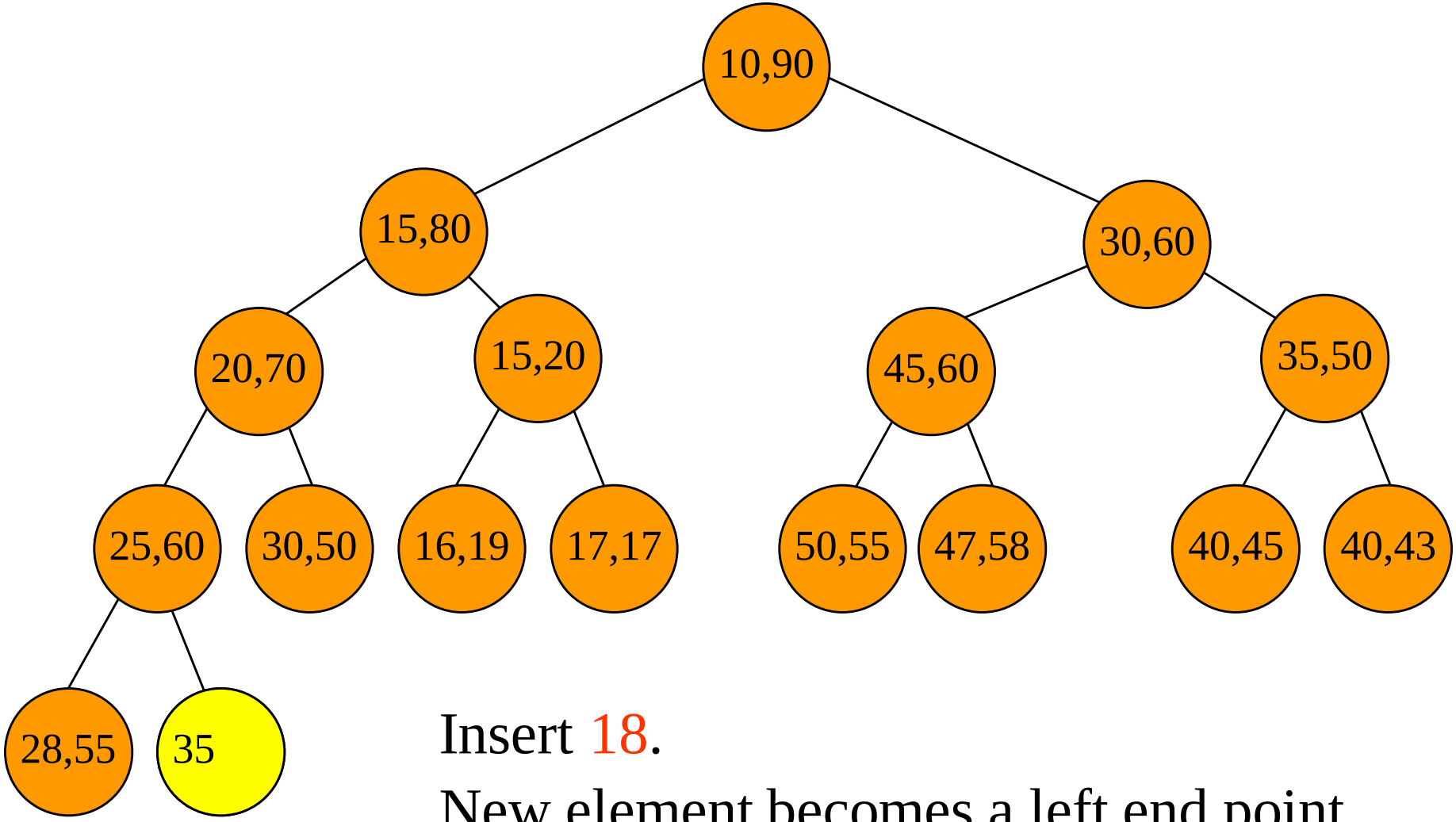


Insert 27.

New element becomes a left end point.

Insert new element into min heap.

Another Insert

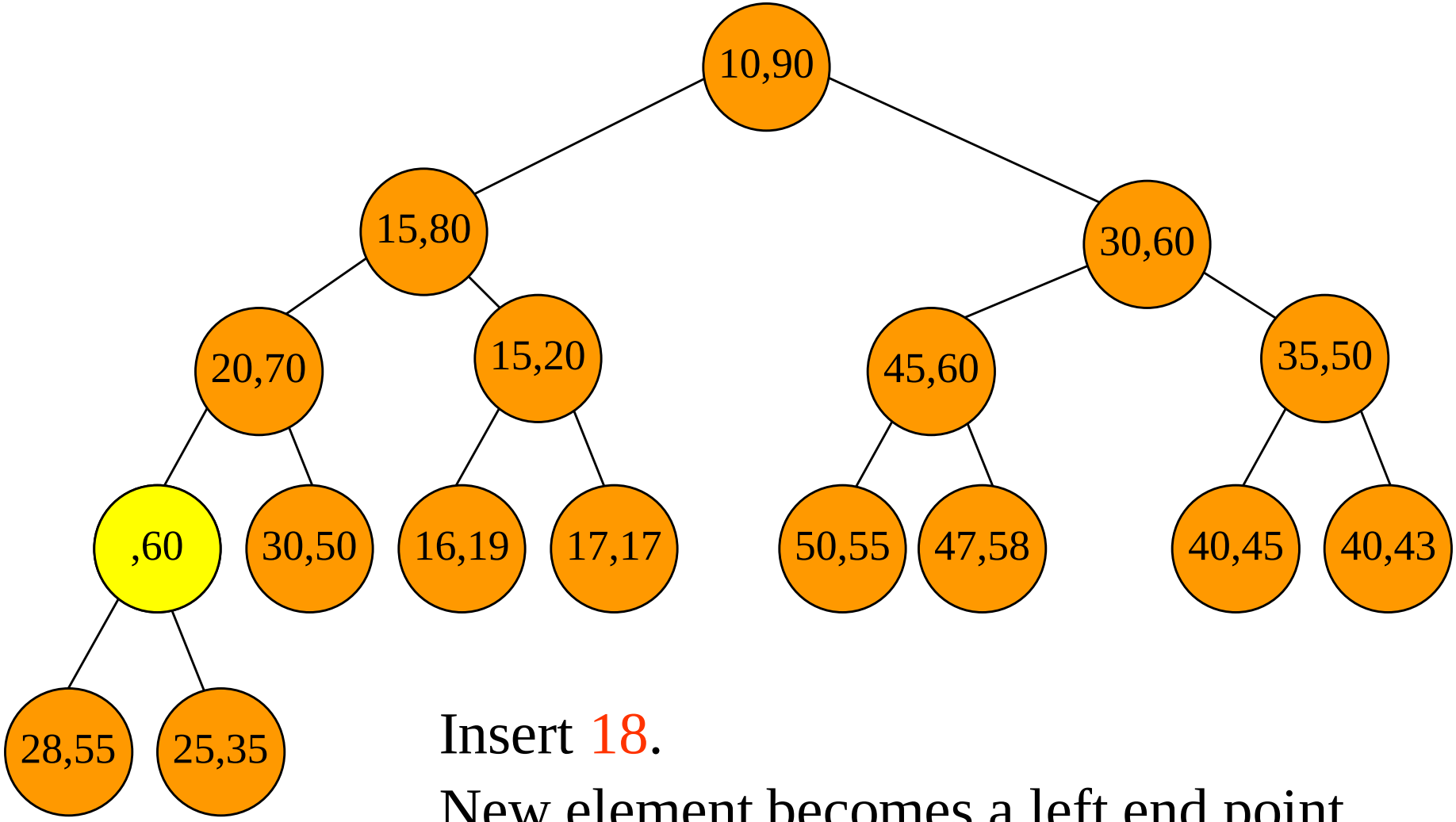


Insert 18.

New element becomes a left end point.

Insert new element into min heap.

Another Insert

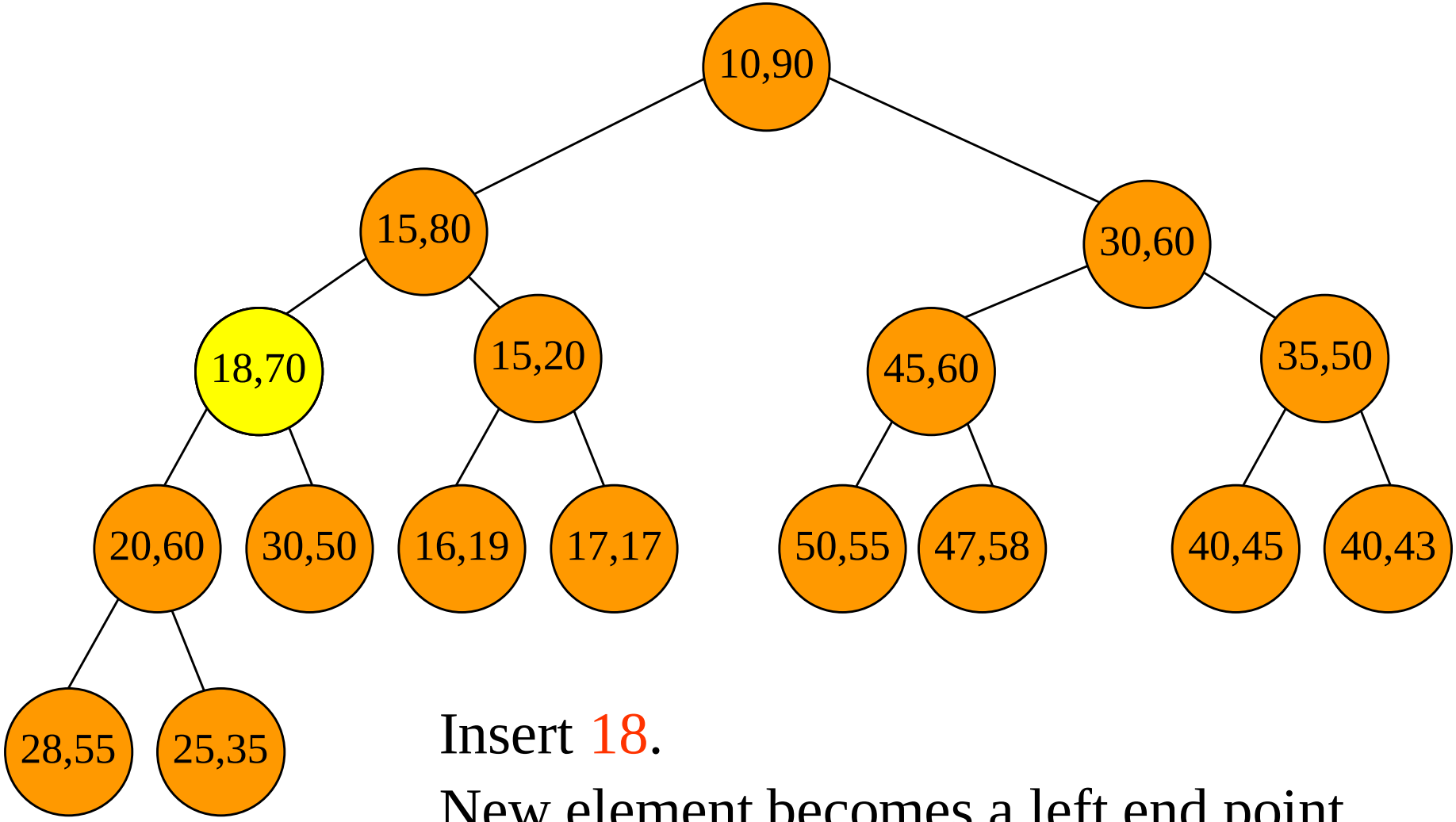


Insert 18.

New element becomes a left end point.

Insert new element into min heap.

Another Insert

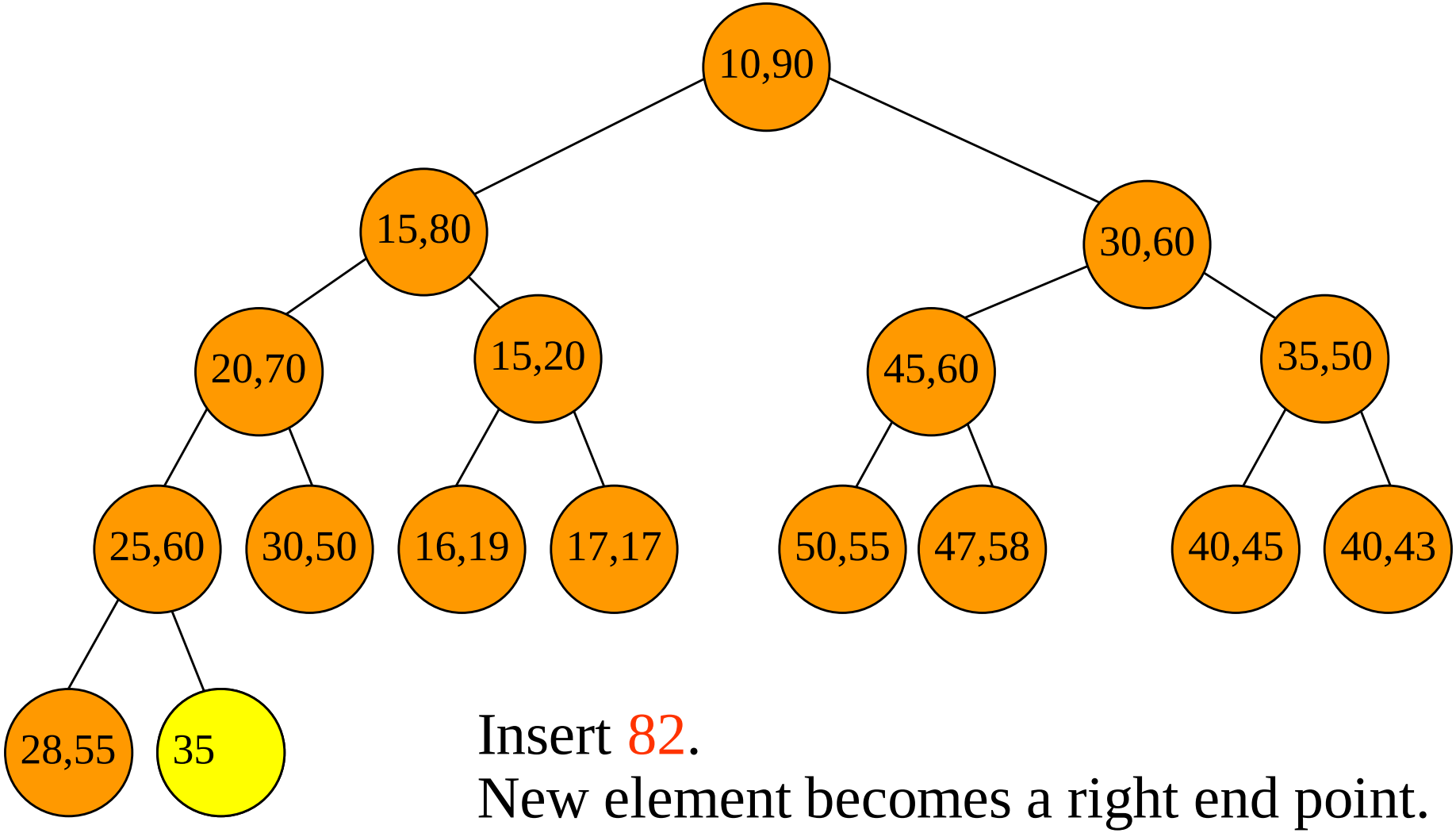


Insert 18.

New element becomes a left end point.

Insert new element into min heap.

Yet Another Insert

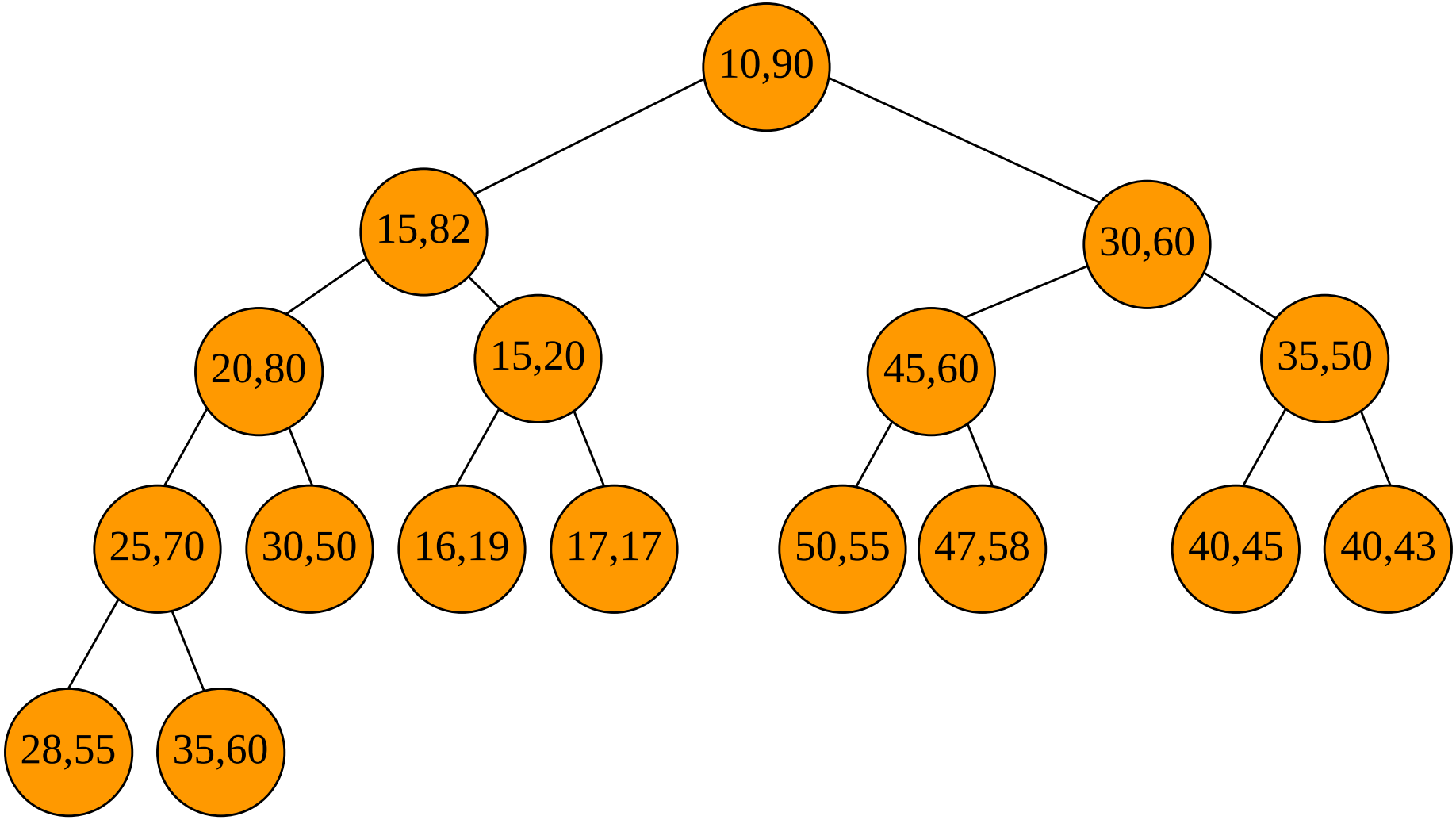


Insert 82.

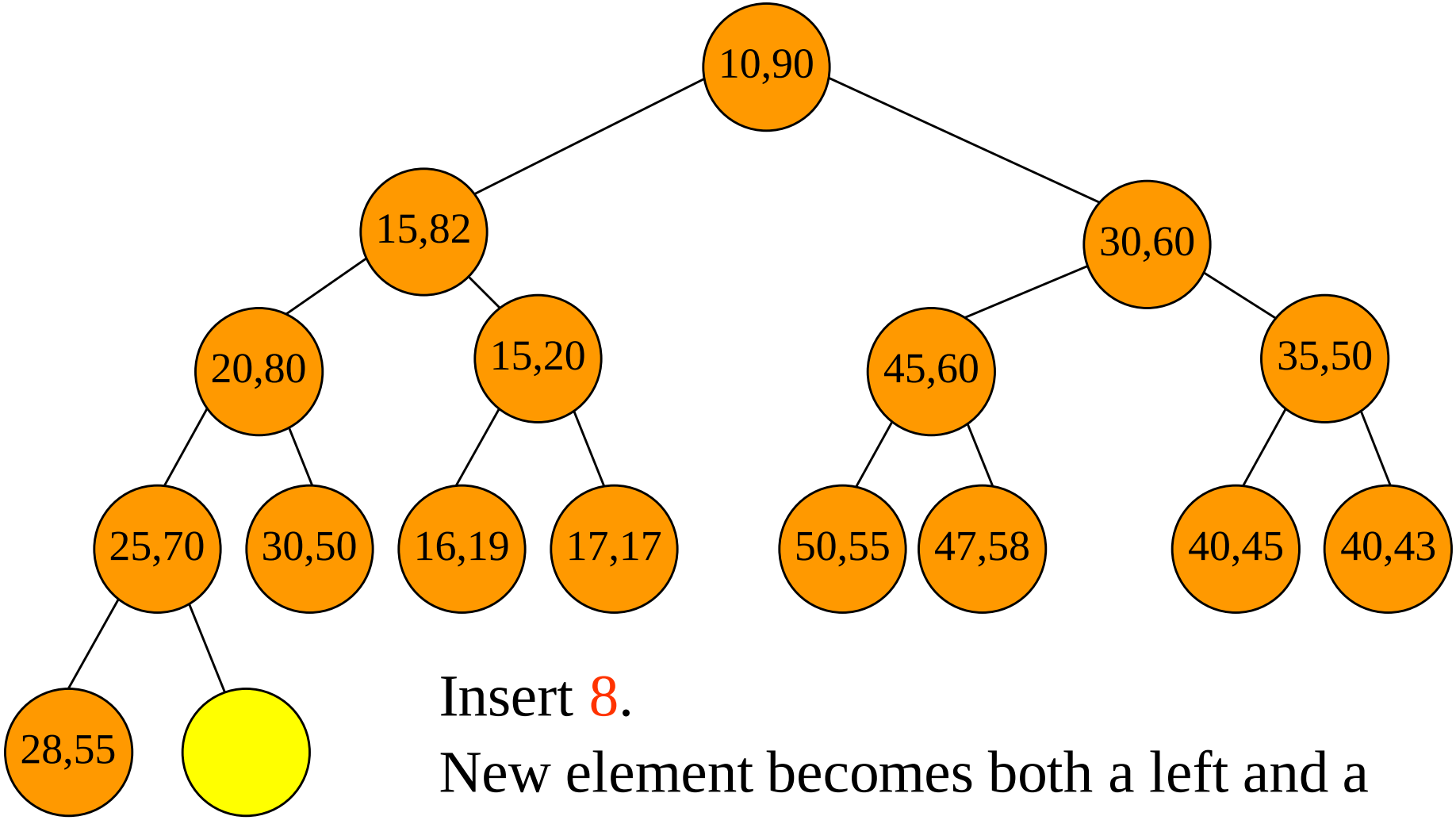
New element becomes a right end point.

Insert new element into max heap.

After 82 Inserted



One More Insert Example

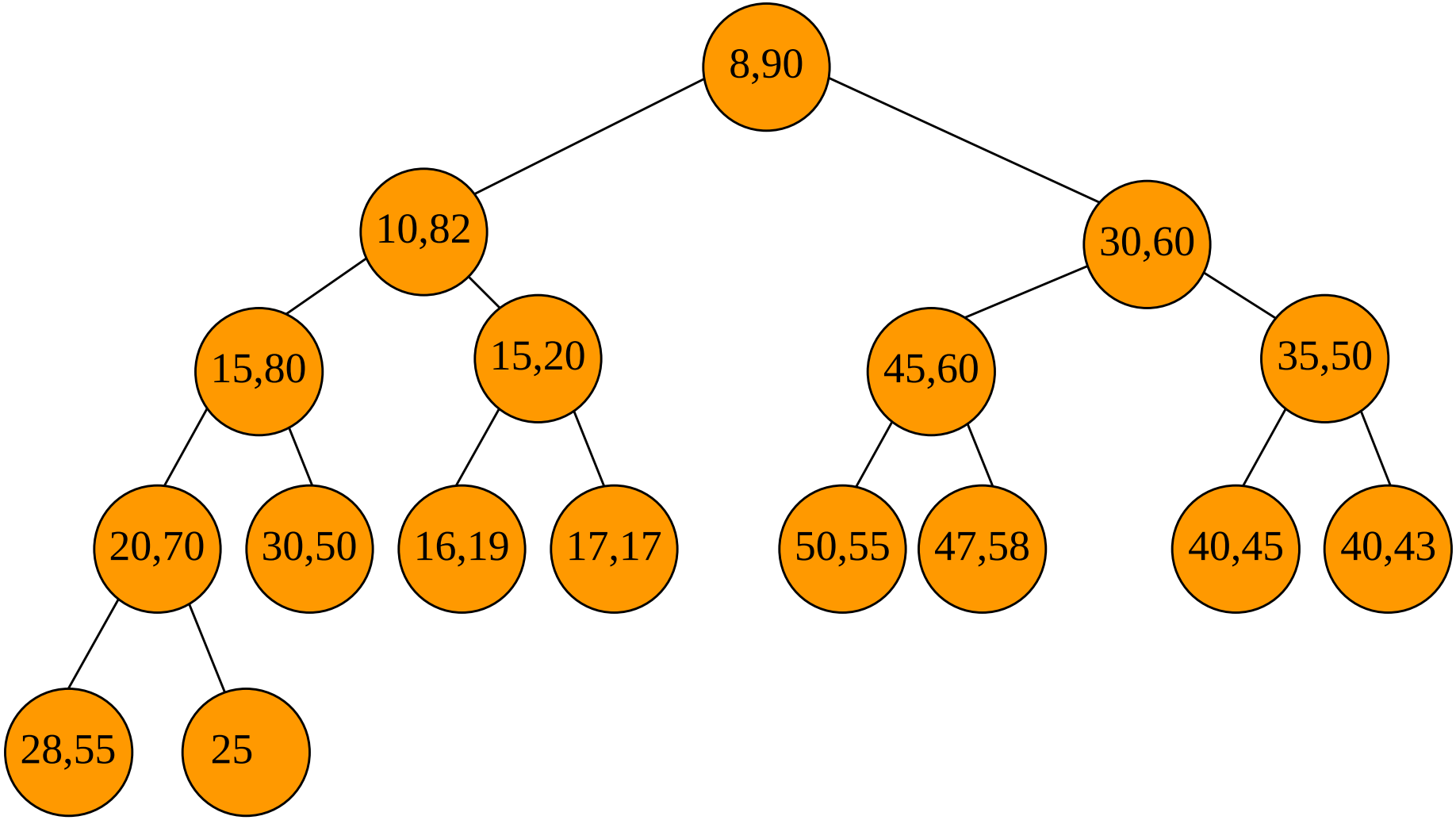


Insert 8.

New element becomes both a left and a right end point.

Insert new element into min heap.

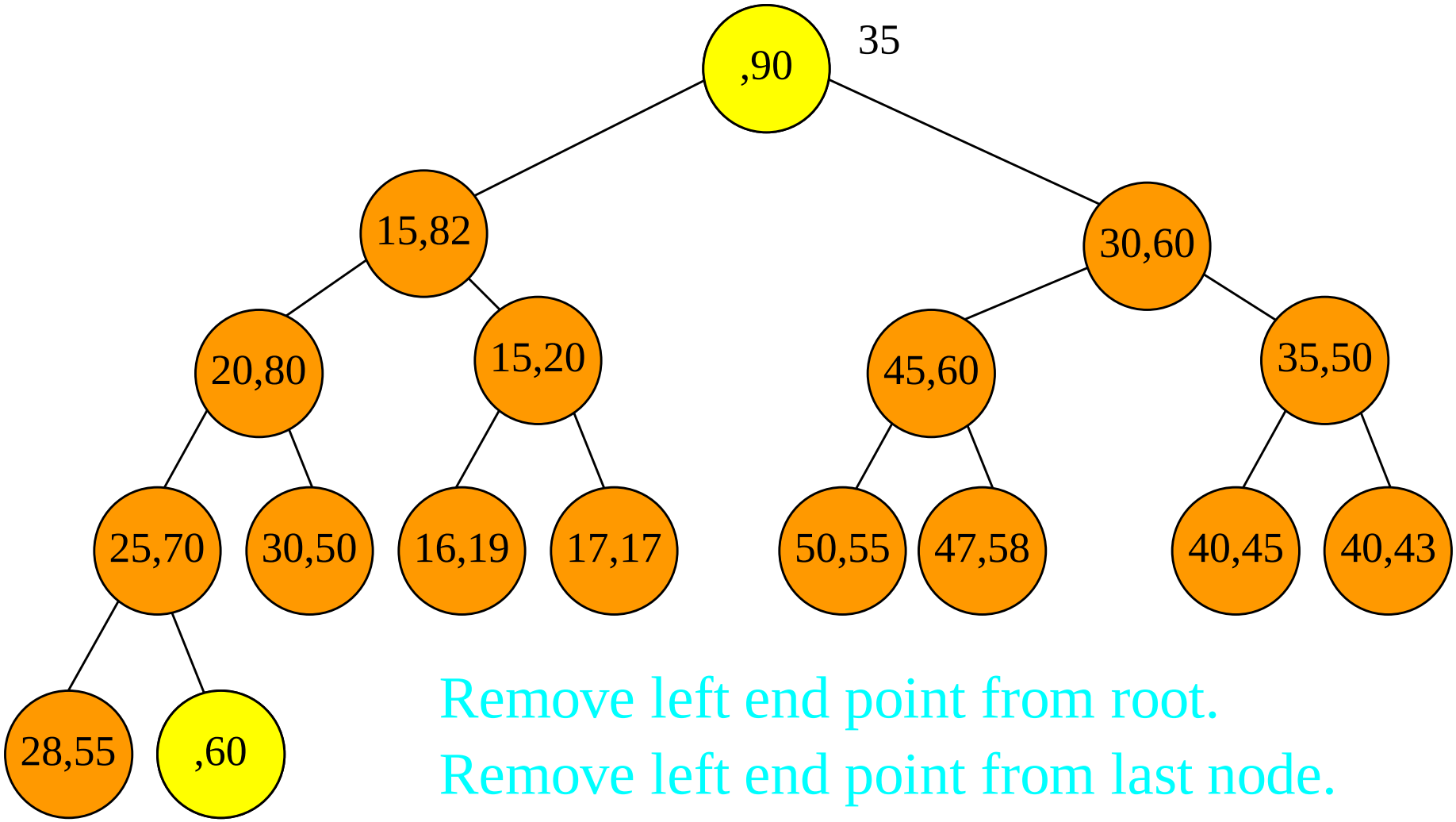
After 8 Is Inserted



Remove Min Element

- n = 0 => fail.
- n = 1 => heap becomes empty.
- n = 2 => only one node, take out left end point.
- n > 2 => not as simple.

Remove Min Element Example



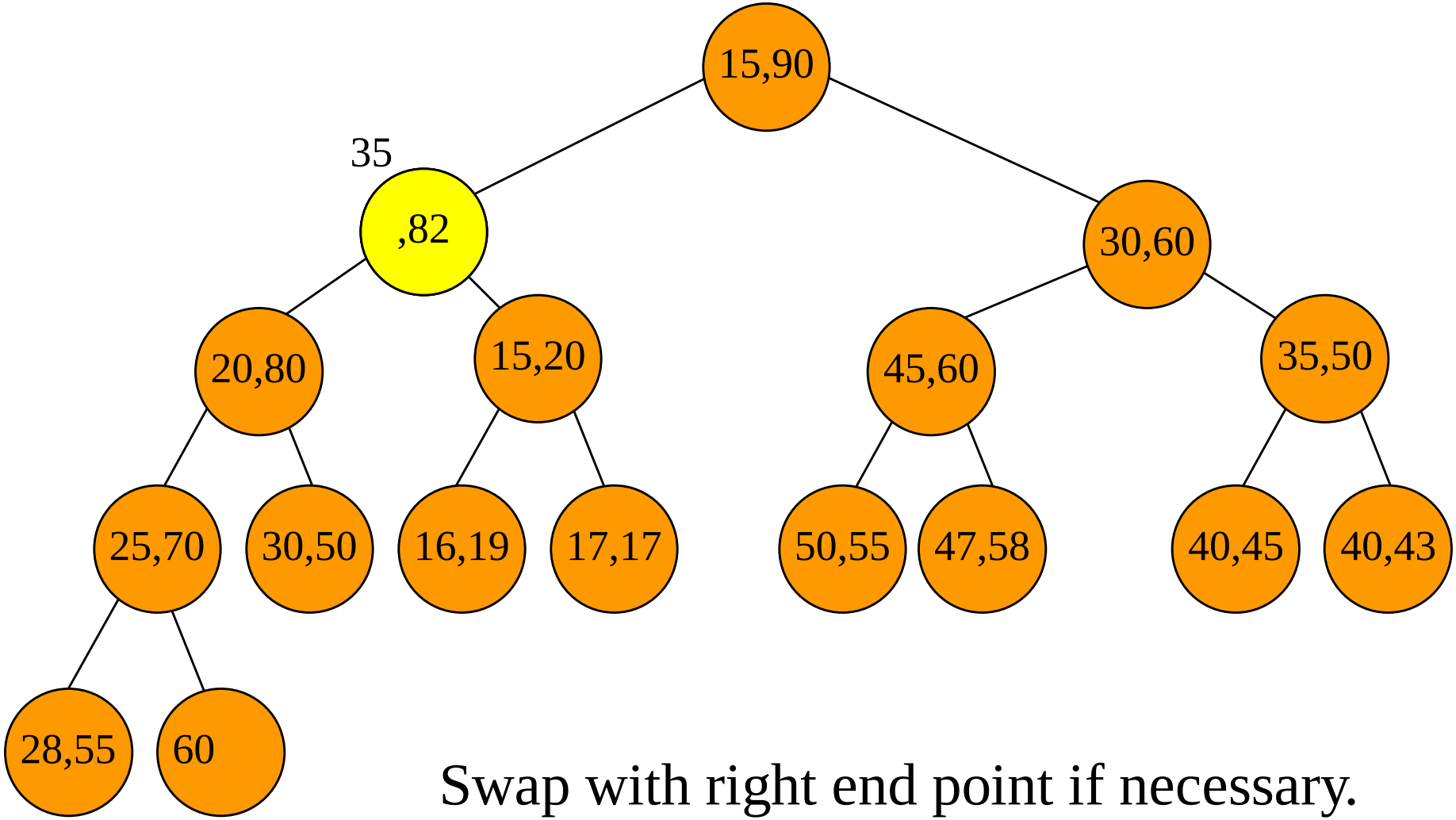
Remove left end point from root.

Remove left end point from last node.

Delete last node if now empty.

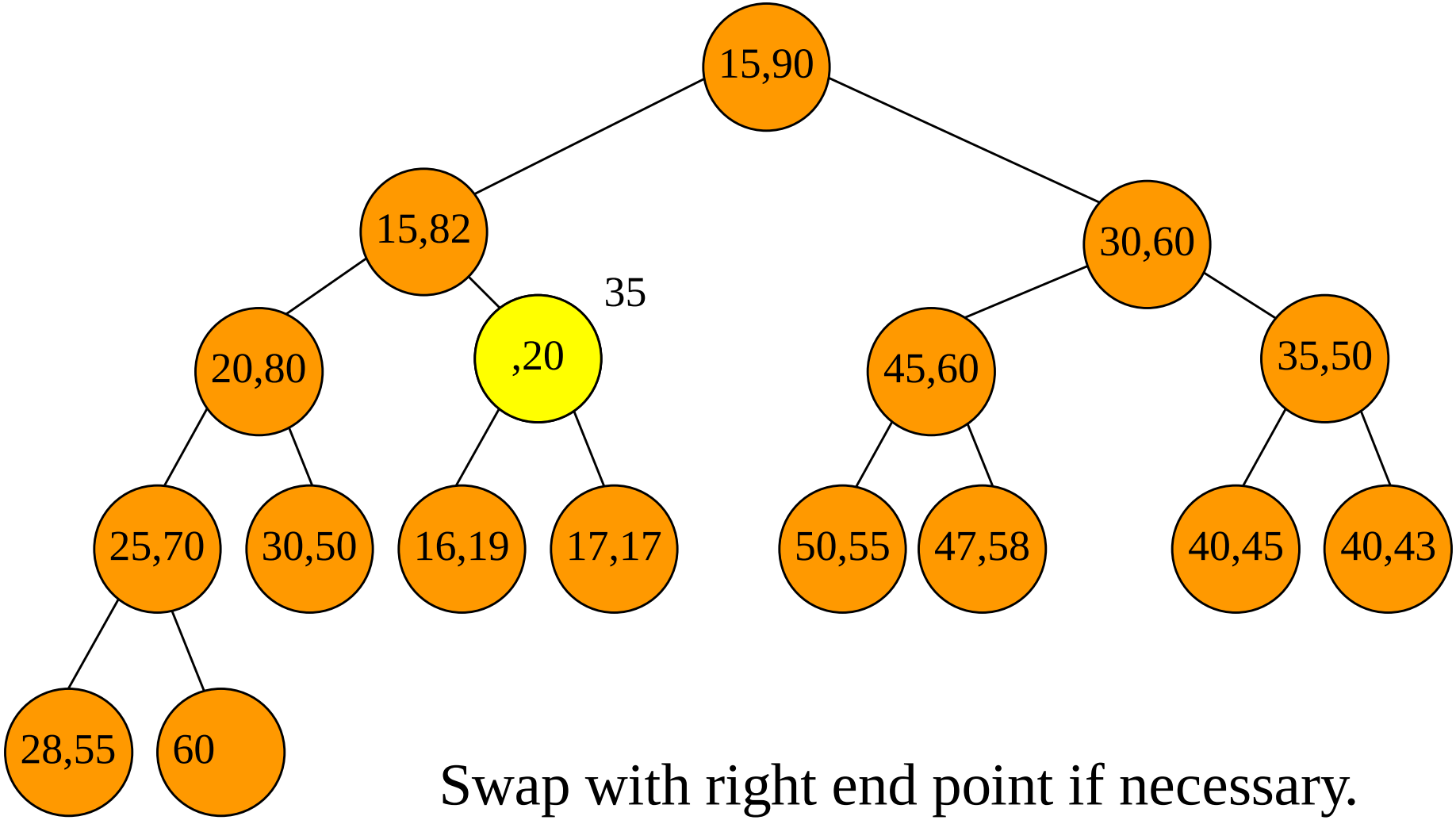
Reinsert into min heap, begin at root.

Remove Min Element Example



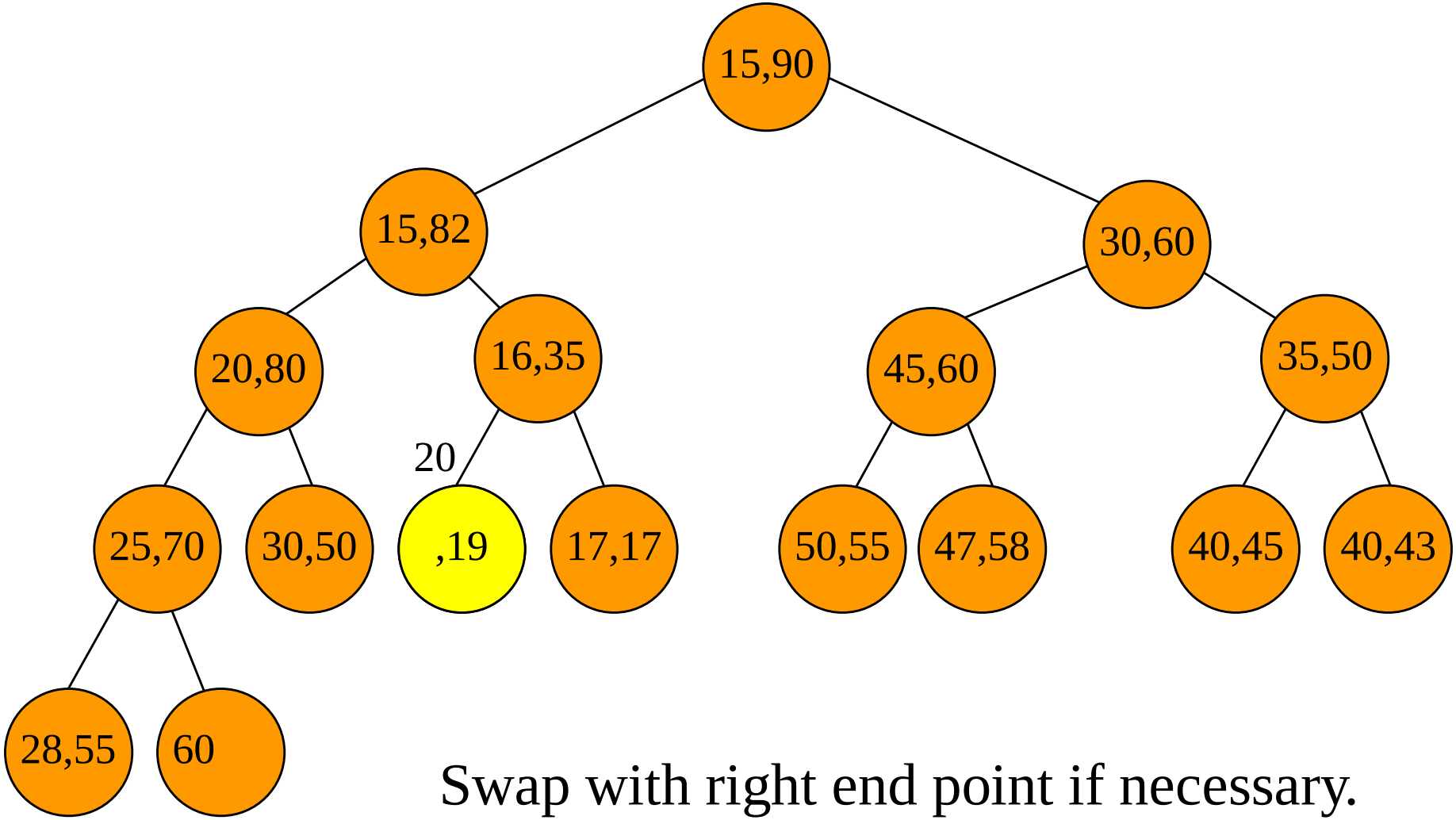
Swap with right end point if necessary.

Remove Min Element Example



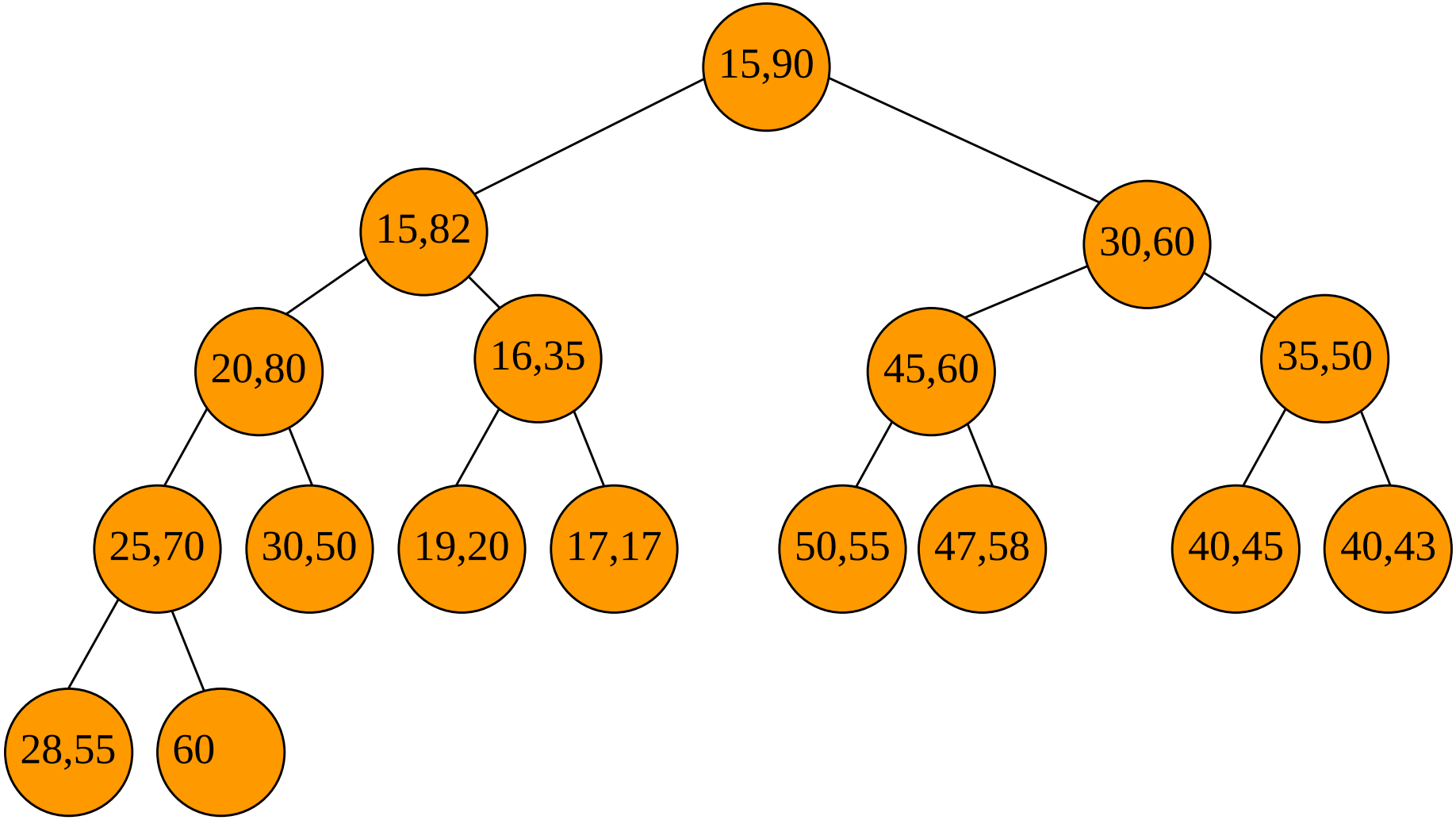
Swap with right end point if necessary.

Remove Min Element Example

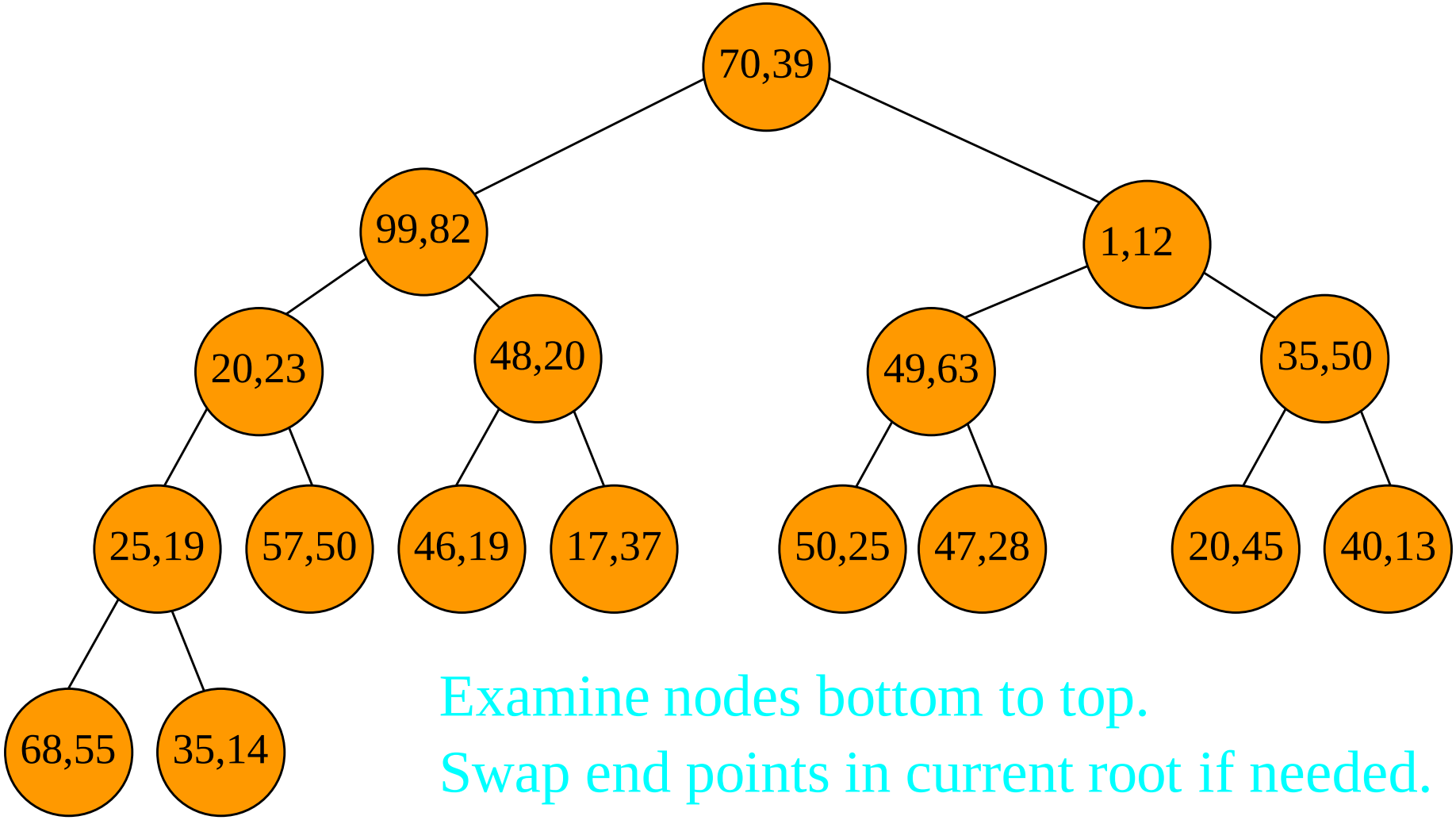


Swap with right end point if necessary.

Remove Min Element Example



Initialize



Examine nodes bottom to top.

Swap end points in current root if needed.

Reinsert left end point into min heap.

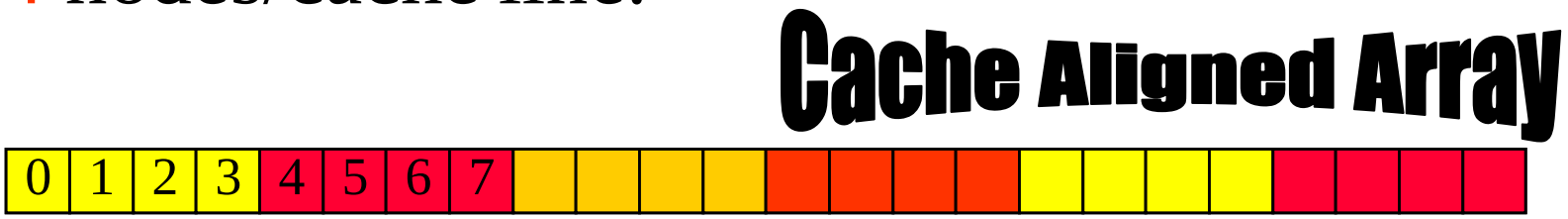
Reinsert right end point into max heap.

Cache Optimization

- Heap operations.
  - Uniformly distributed keys.
  - Insert percolates 1.6 levels up the heap on average.
  - Remove min (max) height – 1 levels down the heap.
- Optimize cache utilization for remove min (max).

Cache Aligned Array

- L1 cache line is 32 bytes.
- L1 cache is 16KB.
- Heap node size is 8 bytes (1 8-byte element).
- 4 nodes/cache line.



- A remove min (max) has ~h L1 cache misses on average.
  - Root and its children are in the same cache line.
  - ~$\log_2 n$ cache misses.
  - Only half of each cache line is used (except root's).

d-ary Heap

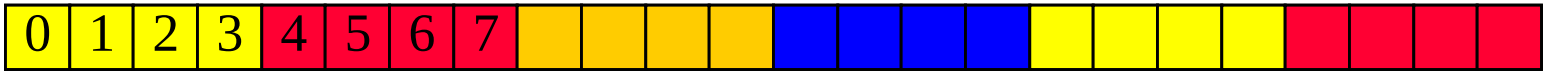
- Complete n node tree whose degree is d.
- Min (max) tree.
- Number nodes in breadth-first manner with root being numbered 1.
- $\text{Parent}(i) = \text{ceil}((i - 1)/d)$.
- Children are $d*(i - 1) + 2, \ldots, \min\{d*i + 1, n\}$.
- Height is $\log_d n$.
- Height of 4-ary heap is half that of 2-ary heap.

d = 4, 4-Heap

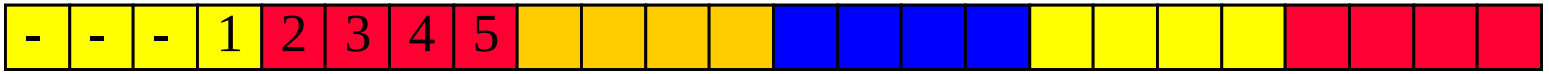
- Worst-case insert moves up half as many levels as when d = 2.
  - Average remains at about 1.6 levels.
- Remove-min operations now do 4 compares per level rather than 2 (determine smallest child and see if this child is smaller than element being relocated).
  - But, number of levels is half.
  - Other operations associated with remove min are halved (move small element up, loop iterations, etc.)

4-Heap Cache Utilization

- Standard mapping into cache-aligned array.



- Siblings are in 2 cache lines.
  - ~$\log_2 n$ cache misses for average remove min (max).
- Shift 4-heap by 2 slots.



- Siblings are in same cache line.
  - ~$\log_4 n$ cache misses for average remove min (max).

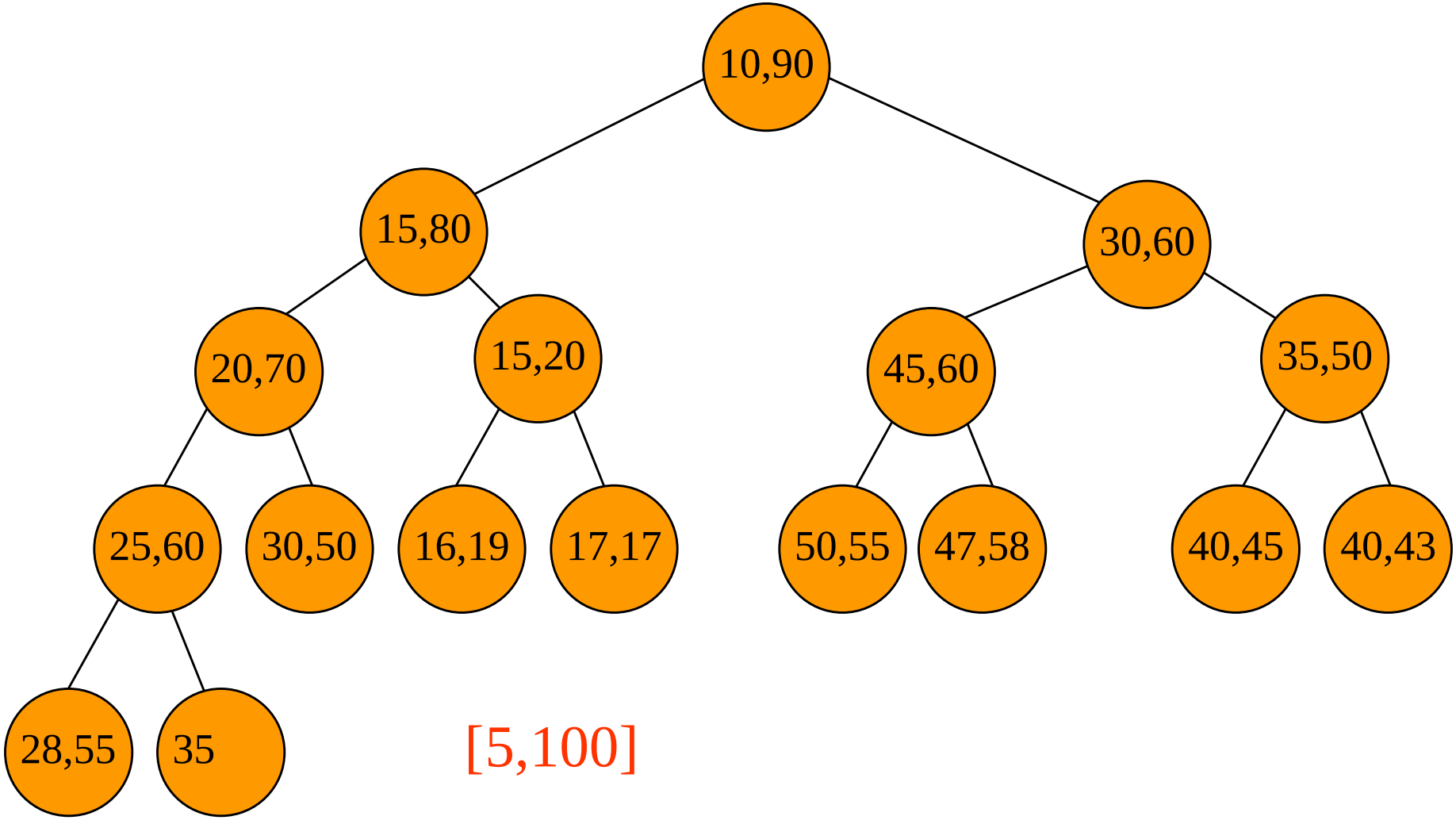
d-ary Heap Performance

- Speedup of about 1.5 to 1.8 when sorting 1 million elements using heapsort and cache-aligned 4-heap vs. 2-heap that begins at array position 0.
- Cache-aligned 4-heap generally performs as well as, or better, than other d-heaps.
- Use degree 4 complete tree for interval heaps instead of degree 2.

Application Of Interval Heaps

- Complementary range search problem.
  - Collection of 1D points (numbers).
  - Insert a point.
    - O(log n)
  - Remove a point given its location in the structure.
    - O(log n)
  - Report all points not in the range [a,b], a <= b.
    - O(k), where k is the number of points not in the range.

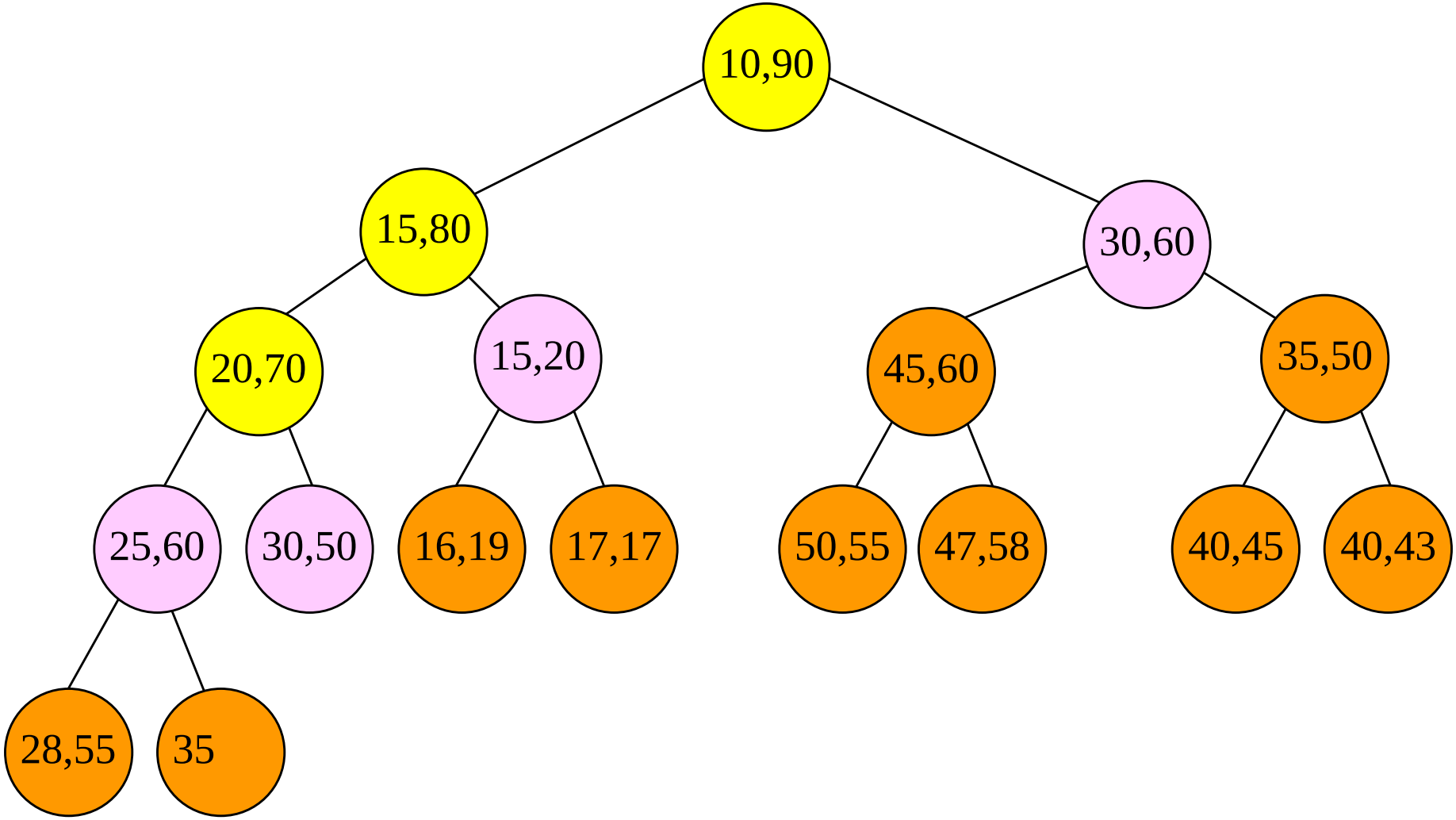
Example



[5,100]

[2,65]

Example


10,90
15,80
30,60
20,70
15,20
45,60
35,50
25,60
30,50
16,19
17,17
50,55
47,58
40,45
40,43
28,55
35

[2,65]

Acknowledgements

- E. Horowitz, S. Sahni, and D. Mehta, Fundamentals of data structures in C++, Second Edition, Silicon Press, 2007.
- Lecture Slides of Prof Sartaj Sahni.